

Guia Completo de Comandos

Do básico ao avançado — tudo que um desenvolvedor precisa dominar

Versão 1.0 · 2025

Este guia cobre todos os comandos Git — desde os primeiros passos até operações avançadas como rebase interativo, bisect, worktrees e hooks — com exemplos práticos e dicas de uso profissional.

Índice de Conteúdo

- 1. Fundamentos do Git** — Instalação, configuração e conceitos
- 2. Repositórios** — Init, clone e estrutura
- 3. Staging & Commits** — Add, commit, amend e boas práticas
- 4. Branches** — Criar, navegar, renomear e deletar
- 5. Merge & Rebase** — Integração de branches
- 6. Repositórios Remotos** — Push, pull, fetch e remotes
- 7. Stash** — Salvar trabalho temporariamente
- 8. Tags** — Versioning com tags
- 9. Log & Histórico** — Explorar o histórico do repositório
- 10. Diff & Comparações** — Inspecionar mudanças
- 11. Reset, Revert & Restore** — Desfazer alterações
- 12. Cherry-pick** — Aplicar commits específicos
- 13. Rebase Interativo** — Reescrever histórico
- 14. Bisect** — Encontrar bugs por busca binária
- 15. Submodules & Subtrees** — Repositórios dentro de repositórios
- 16. Worktrees** — Múltiplos working trees
- 17. Reflog** — Recuperar commits perdidos
- 18. Hooks** — Automação com Git hooks
- 19. Configurações Avançadas** — Aliases, templates e config
- 20. Git Internals** — Objetos, plumbing e porcelain
- 21. Fluxos de Trabalho** — Gitflow, GitHub Flow e trunk-based
- 22. Tabela de Referência Rápida** — Cheatsheet final

1. Fundamentos do Git

Git é um sistema de controlo de versão distribuído criado por Linus Torvalds em 2005. Cada programador possui uma cópia completa do repositório, incluindo todo o histórico. O Git trabalha com três áreas principais: **Working Directory** (ficheiros actuais), **Staging Area / Index** (mudanças preparadas) e **Repository** (commits guardados).

Instalação

```
sudo apt install git
# Linux (Debian/Ubuntu)

brew install git
# macOS com Homebrew

winget install Git.Git
# Windows com winget

git --version
# Verificar versão instalada
```

Configuração Inicial (obrigatória)

```
git config --global user.name "João Paulino"
# Definir nome global

git config --global user.email "joao@email.com"
# Definir email global

git config --global core.editor "code --wait"
# Definir VS Code como editor

git config --global init.defaultBranch main
# Branch padrão = main

git config --global color.ui auto
# Activar cores no terminal

git config --list
# Ver todas as configurações activas

git config --global -e
# Editar config global no editor
```

■ **Dica:** Use `--local` para configurações específicas por repositório, sem o `--global`.

2. Repositórios

```
git init
# Iniciar repositório no directório actual

git init meu-projeto
# Criar pasta e iniciar repositório dentro

git init --bare repo.git
# Repositório bare (para servidores/remotes)

git clone https://github.com/user/repo.git
# Clonar repositório remoto

git clone https://github.com/user/repo.git minha-pasta
# Clonar com nome personalizado

git clone --depth 1 https://github.com/user/repo.git
# Clone superficial (apenas último commit)

git clone --branch develop https://github.com/user/repo.git
# Clonar branch específica

git clone --recurse-submodules https://github.com/user/repo.git
# Clonar com submodules
```

Inspecionar o Repositório

```
git status
# Ver estado actual dos ficheiros

git status -s
# Estado resumido (short)

git status --porcelain
# Saída para scripts/automação
```

3. Staging & Commits

Adicionar ao Staging

```
git add ficheiro.js
# Adicionar ficheiro específico

git add .
# Adicionar tudo no directório actual

git add -A
# Adicionar todas as mudanças (incluindo deletes)

git add -p
# Adicionar interactivamente por blocos (hunks)

git add -u
# Adicionar apenas ficheiros já rastreados

git add '*.css'
# Adicionar por padrão glob
```

Criar Commits

```
git commit -m "feat: adicionar página de login"
# Commit com mensagem

git commit -am 'fix: corrigir bug no header'
# Add + commit (ficheiros rastreados)

git commit --amend
# Alterar o último commit (mensagem ou conteúdo)

git commit --amend --no-edit
# Alterar conteúdo sem mudar mensagem

git commit --amend --author='Nome '
# Alterar autor do último commit

git commit --allow-empty -m "chore: trigger CI"
# Commit vazio
```

■ **Dica:** Siga o padrão *Conventional Commits*: *feat, fix, docs, chore, style, refactor, test, perf*.

Remover do Staging

```
git restore --staged ficheiro.js
# Remover do staging (manter alterações)

git restore ficheiro.js
# Descartar alterações no working directory

git rm ficheiro.js
# Apagar ficheiro e registar remoção

git rm --cached ficheiro.js
# Deixar de rastrear (manter no disco)
```

.gitignore

```
echo 'node_modules/' >> .gitignore
# Ignorar pasta node_modules

git check-ignore -v ficheiro.log
# Verificar qual regra ignora um ficheiro

git ls-files --ignored --exclude-standard
```


4. Branches

```
git branch
# Listar branches locais
git branch -a
# Listar todas (locais + remotas)
git branch -r
# Listar apenas branches remotas
git branch nova-feature
# Criar nova branch
git switch nova-feature
# Mudar para branch existente (moderno)
git checkout nova-feature
# Mudar para branch (clássico)
git switch -c nova-feature
# Criar e mudar para branch (moderno)
git checkout -b nova-feature
# Criar e mudar para branch (clássico)
git branch -m antigo novo
# Renomear branch
git branch -d feature-login
# Deletar branch (seguro – apenas se merged)
git branch -D feature-login
# Forçar deleção de branch
git push origin --delete feature-login
# Deletar branch no remote
git branch --merged
# Listar branches já integradas ao HEAD
git branch --no-merged
# Listar branches ainda não integradas
```

5. Merge & Rebase

Merge

```
git merge feature-login
# Integrar branch na branch actual

git merge --no-ff feature-login
# Merge sem fast-forward (preserva histórico)

git merge --squash feature-login
# Juntar todos os commits em um só

git merge --abort
# Cancelar merge em conflito

git merge -X theirs feature
# Resolver conflitos automaticamente com 'theirs'

git merge -X ours feature
# Resolver conflitos automaticamente com 'ours'

■ Dica: Em projectos grandes, prefira --no-ff para manter rastreabilidade do histórico.
```

Rebase

```
git rebase main
# Reaplicar commits da branch actual em cima de main

git rebase --onto main server client
# Rebase selectivo de commits

git rebase --abort
# Cancelar rebase em curso

git rebase --continue
# Continuar após resolver conflito

git rebase --skip
# Saltar commit com conflito

■■ Atenção: Nunca faça rebase de branches públicas/partilhadas. Reescreve o histórico!
```

Resolução de Conflitos

```
git diff --diff-filter=U
# Ver apenas ficheiros com conflito

git checkout --theirs ficheiro.js
# Aceitar versão do remote

git checkout --ours ficheiro.js
# Aceitar versão local

git mergetool
# Abrir ferramenta visual de merge

git rerere
# Reutilizar resolução de conflitos anteriores
```

6. Repositórios Remotos

```
git remote -v
# Listar remotes com URLs

git remote add origin https://github.com/user/repo.git
# Adicionar remote

git remote rename origin upstream
# Renomear remote

git remote remove origin
# Remover remote

git remote set-url origin https://novo-url.git
# Alterar URL do remote
```

Push

```
git push origin main
# Enviar branch para remote

git push -u origin main
# Push + definir tracking (upstream)

git push --all origin
# Enviar todas as branches

git push --tags origin
# Enviar todas as tags

git push --force origin main
# Forçar push (PERIGOSO)

git push --force-with-lease origin main
# Forçar push (mais seguro)

git push origin --delete feature
# Apagar branch remota
```

■ ■ **Atenção:** Evite `--force`. Use `--force-with-lease` para não sobre escrever trabalho de colegas.

Pull & Fetch

```
git fetch origin
# Baixar mudanças sem integrar

git fetch --all
# Baixar de todos os remotes

git pull origin main
# Fetch + merge

git pull --rebase origin main
# Fetch + rebase (histórico limpo)

git pull --ff-only
# Apenas integrar se for fast-forward
```

7. Stash

O stash guarda temporariamente alterações não commitadas, libertando o working directory.

```
git stash
# Guardar alterações actuais
git stash push -m 'wip: login form'
# Stash com nome descritivo
git stash -u
# Incluir ficheiros não rastreados no stash
git stash -a
# Incluir também ficheiros ignorados
git stash list
# Listar todos os stashes
git stash show stash@{0}
# Ver resumo do stash mais recente
git stash show -p stash@{0}
# Ver diff completo do stash
git stash pop
# Restaurar e apagar o stash mais recente
git stash apply stash@{2}
# Aplicar stash específico sem apagar
git stash drop stash@{1}
# Apagar stash específico
git stash clear
# Apagar todos os stashes
git stash branch nova-branch stash@{0}
# Criar branch a partir de stash
```

8. Tags

```
git tag
# Listar todas as tags

git tag v1.0.0
# Criar tag leve

git tag -a v1.0.0 -m "Release v1.0.0"
# Criar tag anotada (recomendada)

git tag v1.0.0 abc1234
# Criar tag num commit específico

git show v1.0.0
# Ver detalhes da tag

git push origin v1.0.0
# Enviar tag específica

git push origin --tags
# Enviar todas as tags

git tag -d v1.0.0
# Apagar tag local

git push origin --delete v1.0.0
# Apagar tag remota

git checkout v1.0.0
# Ir para o estado de uma tag (detached HEAD)
```

■ **Dica:** Prefira sempre tags anotadas (-a) em releases: contêm metadata como data e autor.

9. Log & Histórico

```
git log
# Histórico completo
git log --oneline
# Uma linha por commit
git log --oneline --graph --all
# Grafo visual de todas as branches
git log --oneline --graph --decorate --all
# Grafo com referências
git log -10
# Últimos 10 commits
git log -p
# Histórico com diff de cada commit
git log --stat
# Histórico com estatísticas de ficheiros
git log --author='João'
# Filtrar por autor
git log --since='2024-01-01' --until='2024-12-31'
# Filtrar por data
git log --grep='feat'
# Filtrar por texto na mensagem
git log -S 'nomeFuncao'
# Encontrar quando código foi adicionado/removido
git log --follow ficheiro.js
# Histórico seguindo renomeações
git log main..feature
# Commits em feature que não estão em main
git log --format='%H %an %s'
# Formato personalizado
git shortlog -sn
# Resumo de commits por autor
```

10. Diff & Comparações

```
git diff
# Diferença entre working dir e staging
git diff --staged
# Diferença entre staging e último commit
git diff HEAD
# Diferença entre working dir e último commit
git diff main feature
# Comparar dois branches
git diff v1.0 v2.0
# Comparar duas tags
git diff abc1234 def5678
# Comparar dois commits
git diff HEAD~3
# Comparar com 3 commits atrás
git diff --stat
# Apenas estatísticas (sem conteúdo)
git diff --name-only
# Apenas nomes de ficheiros modificados
git diff --word-diff
# Diff por palavra (útil para texto)
git show abc1234
# Ver conteúdo de um commit específico
git show HEAD:ficheiro.js
# Ver ficheiro no estado do HEAD
git blame ficheiro.js
# Ver quem escreveu cada linha
git blame -L 10,25 ficheiro.js
# Blame de linhas 10 a 25 apenas
```

11. Reset, Revert & Restore

git reset — Mover o ponteiro HEAD

```
git reset HEAD~1
# Desfazer último commit (manter staged)

git reset --soft HEAD~1
# Desfazer commit, manter staging e ficheiros

git reset --mixed HEAD~1
# Desfazer commit e staging (default)

git reset --hard HEAD~1
# Desfazer tudo — IRREVERSÍVEL!

git reset HEAD ficheiro.js
# Remover ficheiro do staging
```

■ **Atenção:** *--hard destrói trabalho não commitado permanentemente. Use com cuidado!*

git revert — Reverter com novo commit (seguro)

```
git revert abc1234
# Criar commit que inverte um commit específico

git revert HEAD
# Reverter o último commit

git revert HEAD~3..HEAD
# Reverter os últimos 3 commits

git revert --no-commit abc1234
# Reverter sem criar commit imediatamente
```

■ **Dica:** *Prefira revert em branches públicas — não reescreve histórico.*

git restore — Restaurar ficheiros

```
git restore ficheiro.js
# Descartar mudanças no working directory

git restore --staged ficheiro.js
# Remover do staging sem apagar mudanças

git restore --source=HEAD~2 ficheiro.js
# Restaurar ficheiro de commit anterior

git restore .
# Descartar todas as mudanças locais
```

12. Cherry-pick

Cherry-pick permite aplicar um commit específico de outra branch sem fazer merge completo.

```
git cherry-pick abc1234
# Aplicar commit específico na branch actual
git cherry-pick abc1234 def5678
# Aplicar múltiplos commits
git cherry-pick abc1234..def5678
# Aplicar intervalo de commits
git cherry-pick --no-commit abc1234
# Aplicar sem criar commit (fica em staging)
git cherry-pick -x abc1234
# Incluir referência ao commit original na mensagem
git cherry-pick --abort
# Cancelar cherry-pick em conflito
git cherry-pick --continue
# Continuar após resolver conflito
```

13. Rebase Interativo

O rebase interativo é uma das ferramentas mais poderosas do Git. Permite reordenar, editar, juntar e apagar commits do histórico.

```
git rebase -i HEAD~5
# Rebase interativo dos últimos 5 commits

git rebase -i abc1234
# Rebase interativo a partir de commit específico

git rebase -i --root
# Rebase interativo de todo o repositório
```

Acções disponíveis no editor interativo

Comando	Descrição
<code>pick / p</code>	Manter o commit sem alteração
<code>reword / r</code>	Manter mas editar a mensagem do commit
<code>edit / e</code>	Pausar para editar o commit (conteúdo)
<code>squash / s</code>	Juntar com commit anterior (manter mensagens)
<code>fixup / f</code>	Juntar com commit anterior (descartar mensagem)
<code>drop / d</code>	Apagar o commit completamente
<code>exec / x</code>	Executar um comando shell após o commit
<code>break / b</code>	Pausar o rebase nesse ponto

■ ■ **Atenção:** *Rebase interativo reescreve o histórico — nunca use em branches partilhadas!*

14. Git Bisect

Bisect usa busca binária para encontrar o commit que introduziu um bug. Git divide o histórico ao meio, você testa, e vai reduzindo até isolar o culpado.

```
git bisect start
# Iniciar sessão de bisect

git bisect bad
# Marcar commit actual como 'com bug'

git bisect good v1.0.0
# Marcar versão conhecida como 'sem bug'

git bisect bad
# Marcar commit actual como 'mau' (repetir até encontrar)

git bisect good
# Marcar como 'bom' e avançar

git bisect skip
# Saltar commit que não pode ser testado

git bisect reset
# Terminar sessão e voltar ao HEAD original

git bisect run npm test
# Automação: bisect com script de teste
```

■ **Dica:** Com um repositório de 1000 commits, bisect encontra o bug em apenas ~10 passos!

15. Submodules & Subtrees

Submodules — Repositório dentro de repositório

```
git submodule add https://github.com/user/lib.git libs/lib
# Adicionar submodule

git submodule init
# Inicializar submodules após clonar

git submodule update
# Actualizar submodules para o commit referenciado

git submodule update --init --recursive
# Init + update recursivo

git submodule update --remote
# Actualizar para o commit mais recente do remote

git submodule foreach git pull origin main
# Pull em todos os submodules

git submodule deinit libs/lib
# Desactivar submodule

git rm libs/lib
# Remover submodule do repositório
```

Subtrees — Alternativa mais simples aos submodules

```
git subtree add --prefix=libs/lib https://github.com/user/lib.git main
# Adicionar subtree

git subtree pull --prefix=libs/lib https://github.com/user/lib.git main
# Actualizar subtree

git subtree push --prefix=libs/lib https://github.com/user/lib.git main
# Push de mudanças ao projecto original
```

■ **Dica:** Subtrees são mais simples para colaboradores — não requerem `git submodule init`.

16. Git Worktrees

Worktrees permitem ter múltiplas branches activas em simultâneo em directorias diferentes, sem precisar de stash ou de fazer clone múltiplo.

```
git worktree add ../hotfix hotfix/login-bug
# Criar worktree para branch existente

git worktree add -b nova-feature ../feature-dir main
# Criar worktree com nova branch

git worktree list
# Listar todos os worktrees activos

git worktree prune
# Remover referências a worktrees apagados

git worktree remove ../hotfix
# Remover worktree específico
```

■ **Dica:** *Ideal para trabalhar numa hotfix urgente sem perder o progresso actual.*

17. Reflog — Recuperar Commits Perdidos

O reflog regista todas as movimentações do HEAD, mesmo operações destrutivas. É o 'plano B' definitivo do Git.

```
git reflog
# Ver histórico completo de movimentos do HEAD
```

```
git reflog --all
# Reflog de todas as referências
```

```
git reflog show feature-branch
# Reflog de branch específica
```

```
git checkout HEAD@{3}
# Ir para estado de 3 movimentos atrás
```

```
git reset --hard HEAD@{5}
# Restaurar para estado de 5 acções atrás
```

```
git branch recuperada HEAD@{4}
# Criar branch a partir de ponto no reflog
```

■ **Dica:** O reflog mantém histórico por 90 dias por padrão. Use-o para recuperar resets acidentais!

18. Git Hooks

Hooks são scripts executados automaticamente antes/depois de eventos Git. Ficam em `.git/hooks/` e podem ser escritos em bash, Python, Node.js, etc.

Hooks do lado do cliente

Comando	Descrição
<code>pre-commit</code>	Antes de criar commit — ideal para linting e testes
<code>prepare-commit-msg</code>	Antes de abrir editor de mensagem
<code>commit-msg</code>	Validar formato da mensagem do commit
<code>post-commit</code>	Após commit — notificações, logs
<code>pre-push</code>	Antes de push — rodar testes completos
<code>post-checkout</code>	Após mudar de branch
<code>post-merge</code>	Após merge — instalar dependências, etc.
<code>pre-rebase</code>	Antes de iniciar rebase

Exemplo: hook pre-commit com linting

```
chmod +x .git/hooks/pre-commit
# Tornar o script executável

#!/bin/sh npx eslint . --ext .js,.ts if [ $? -ne 0 ]; then echo 'ESLint falhou. Commit
bloqueado.' exit 1 fi
```

■ **Dica:** Use o pacote *'husky'* para gerir hooks facilmente em projectos Node.js.

19. Configurações Avançadas & Aliases

Aliases úteis

```
git config --global alias.st status
# git st → git status

git config --global alias.co checkout
# git co → git checkout

git config --global alias.br branch
# git br → git branch

git config --global alias.lg "log --oneline --graph --all --decorate"
# git lg → grafo bonito

git config --global alias.undo "reset --soft HEAD~1"
# git undo → desfazer último commit

git config --global alias.alias "config --get-regexp alias"
# git alias → listar aliases
```

Configurações úteis

```
git config --global pull.rebase true
# Pull usa rebase por padrão

git config --global push.autoSetupRemote true
# Push cria tracking automaticamente

git config --global rebase.autoStash true
# Stash automático antes de rebase

git config --global core.autocrlf input
# Normalizar line endings (Linux/Mac)

git config --global help.autocorrect 20
# Correção automática de typos (2 seg)

git config --global diff.colorMoved zebra
# Colorir linhas movidas no diff

git config --global merge.conflictstyle diff3
# Mostrar versão base nos conflitos
```

20. Git Internals — Plumbing & Porcelain

Git internamente armazena tudo como objectos imutáveis: blobs (conteúdo de ficheiros), trees (estrutura de directórios), commits e tags. Compreender isto ajuda a depurar problemas complexos.

Comandos de baixo nível (plumbing)

```
git cat-file -t abc1234
# Ver tipo de objecto (blob/tree/commit/tag)

git cat-file -p abc1234
# Ver conteúdo de qualquer objecto

git ls-files
# Listar ficheiros no index/staging

git ls-tree HEAD
# Listar tree do commit actual

git rev-parse HEAD
# Obter hash completo do HEAD

git rev-parse --short HEAD
# Hash curto do HEAD

git count-objects -v
# Estatísticas de objectos no repositório

git gc
# Garbage collection – otimizar repositório

git fsck
# Verificar integridade do repositório

git verify-pack -v .git/objects/pack/*.idx
# Inspeccionar pack files
```

Manutenção

```
git prune
# Remover objectos não referenciados

git pack-refs --all
# Compactar referências

git clean -fd
# Apagar ficheiros não rastreados e directorias

git clean -fdn
# Simulação: ver o que seria apagado

git clean -fdx
# Apagar incluindo ficheiros ignorados
```

21. Fluxos de Trabalho

Gitflow

Gitflow usa branches permanentes main e develop, mais branches temporárias: feature/*, release/* e hotfix/*. Ideal para projectos com releases planeadas.

```
git flow init
# Iniciar gitflow (requer git-flow instalado)
git flow feature start nova-funcao
# Iniciar feature branch
git flow feature finish nova-funcao
# Terminar e fazer merge da feature
git flow release start v1.2.0
# Iniciar release branch
git flow hotfix start fix-critico
# Iniciar hotfix
```

GitHub Flow (simplificado)

Uma branch main sempre deployável. Cada feature vai para uma branch, depois Pull Request, revisão e merge. Simples e eficaz para CI/CD contínuo.

```
git switch -c feature/minha-funcao
# 1. Criar branch
git push -u origin feature/minha-funcao
# 2. Push e criar tracking
# Abrir Pull Request no GitHub
# 3. PR para revisão
git switch main && git pull
# 4. Após merge, actualizar main
```

Trunk-Based Development

Todos commitam directamente na main (ou branches de curta duração < 1 dia). Requer feature flags e CI/CD robusto. Usado por Google e Facebook.

22. Tabela de Referência Rápida

Comando	Descrição
<code>git init</code>	Iniciar repositório
<code>git clone</code>	Clonar repositório
<code>git status</code>	Ver estado actual
<code>git add .</code>	Adicionar tudo ao staging
<code>git commit -m 'mensagem'</code>	Criar commit
<code>git push origin main</code>	Enviar para remote
<code>git pull origin main</code>	Baixar e integrar do remote
<code>git fetch origin</code>	Baixar sem integrar
<code>git branch -b nova</code>	Criar e mudar de branch
<code>git merge feature</code>	Integrar branch
<code>git rebase main</code>	Rebase na main
<code>git stash</code>	Guardar trabalho temporário
<code>git stash pop</code>	Restaurar stash
<code>git tag v1.0.0</code>	Criar tag
<code>git log --oneline --graph</code>	Ver histórico visual
<code>git diff --staged</code>	Ver diff no staging
<code>git reset --soft HEAD~1</code>	Desfazer último commit (manter staged)
<code>git revert HEAD</code>	Reverter último commit (seguro)
<code>git cherry-pick abc1234</code>	Aplicar commit específico
<code>git rebase -i HEAD~3</code>	Rebase interactivo (últimos 3)
<code>git bisect start</code>	Iniciar busca binária de bug
<code>git reflog</code>	Ver histórico completo do HEAD
<code>git stash branch nova</code>	Branch a partir de stash
<code>git worktree add ../dir branch</code>	Criar worktree
<code>git clean -fd</code>	Apagar ficheiros não rastreados
<code>git gc</code>	Optimizar repositório